
The Alpha Model, Explained

What it predicts, how it is trained, and how four AI agents train it

This volume explains the quantitative desk's return-prediction model in ordinary language. It assumes you are comfortable with averages, standard deviations, correlation and the idea of fitting a line to data, and it assumes nothing at all about finance or machine learning. Every technical term is defined the first time it appears, and the terms are collected again in a glossary at the end.

The other volumes state the same material formally: the *Quantitative & ML Models* volume gives the equations, and the *glib Integration* volume gives the engineering. This one is the way in.

Contents

1	The question	2
2	The data: a panel	2
3	The single most important rule	3
4	Making companies comparable	3
5	What “training” actually means	4
6	The thing that goes wrong: overfitting	4
7	Splitting the data honestly	5
8	The subtle leak: why a gap is needed	5
9	Walk-forward: the closest thing to a rehearsal	6
10	Skill	6
11	Saying how sure we are	7
12	Beyond skill: is it usable, and is it really alpha?	7
13	Breaking it on purpose	8
14	Where does it work?	8
15	Why the old way was not enough	9
16	What an “agent” is here	9
17	LangGraph: wiring the agents together	10
18	One round, step by step	10
19	Two things the AI is not allowed to do	10
20	Choosing a winner, and when not to	12
21	What you get out	12
	Glossary	13

Part I What the model is trying to do

1 The question

Imagine you have five thousand companies in front of you and one question: over the next year, which of them will do better than the others?

Notice what the question is *not*. It is not “will Apple’s share price go up?” and it is not “will the market crash?”. It is a question about **relative** performance — about ordering the companies from best to worst. This distinction has a name.

Cross-sectional means *comparing many things to each other at a single moment in time*. A *cross-section* is one such snapshot: all five thousand companies as they stood on 31 January 2024. The opposite is **time-series**, which follows one thing as it changes over time. Our model is cross-sectional. Every month it looks at the whole snapshot and sorts the companies. It never tries to say whether the market as a whole will rise.

Why sort rather than forecast a number? Because sorting is both easier and more useful. Easier, because you can be wrong about the general level of returns and still be right about the ordering — if a model says A will return 8% and B will return 3%, and in reality they return −2% and −7%, the *numbers* were badly wrong but the *ordering* was perfect, and a portfolio that bought A and avoided B made money. More useful, because portfolio construction consumes an ordering: you buy the top of the list and avoid the bottom.

The word “alpha”

In finance, **alpha** means return that is *not* explained by simply being exposed to the market. If the market rises 10% and your portfolio rises 10%, you earned no alpha — you could have got that by buying an index fund. If the market rises 10% and you earn 14% with the same amount of risk, the extra 4% is alpha. The name comes from the symbol α for the intercept in a regression, which is where the unexplained part shows up. An “alpha model” is a model whose job is to find that unexplained part.

2 The data: a panel

The model learns from a **panel**, which is a table with two index dimensions instead of one. An ordinary table has one row per thing. A panel has one row per *thing per period*:

Month	Company	Features (inputs)				Label
		book/market	ROE	sales growth	...	
2019-01	AAPL	0.21	0.49	−0.02	...	+0.86
2019-01	JPM	0.78	0.13	+0.04	...	+0.21
2019-01	...					
2019-02	AAPL	0.20	0.49	−0.02	...	+0.79

Three pieces of vocabulary, all visible in that table.

A **feature** (also called a *factor* or a *predictor*) is one measured property of a company that we feed into the model as input. “Book-to-market” — the accounting value of the company divided by its stock-market value — is a feature. So is return on equity. The model currently uses about 66 of them, all drawn from regulatory filings.

A **label** (also called the *target* or the *response*) is the thing we are trying to predict. Here it is the company’s total return over the next 1, 3, 6 or 12 months, depending on which model you are looking at.

Supervised learning is the name for this whole setup: learning from examples where you can see both

the inputs and the correct answer. It is “supervised” in the sense that the historical answer supervises the learning.

The features are grouped into four families, which is just a convenient way of talking about what kind of thing each one measures:

Family	Roughly, it asks
value	Is this company cheap relative to what it owns or earns?
quality	Is this a good business — profitable, efficient, converting profit to cash?
growth	Is it getting bigger, and is it spending to get bigger?
market factor	How has the stock itself behaved — volatile, trending, sensitive to the market?

3 The single most important rule

Everything in this document is downstream of one rule, so it is worth stating before anything else.

Never let the model see the future

When the model makes a prediction for January 2019, it must use only information that was genuinely available in January 2019. Not information we happen to know now.

This sounds obvious and is violated constantly, usually by accident. The failure has a name.

Lookahead bias is any leakage of future information into a decision that was supposed to be made in the past. A model with lookahead bias produces spectacular results that evaporate the moment it is used for real, because in reality the future information is not there.

Two examples of where it creeps in, both of which we guard against explicitly.

Reporting delay. A company’s results for the quarter ending 31 December are not published on 31 December. They appear weeks or months later. If we attached December’s accounting figures to a December prediction, the model would be using numbers nobody had yet. We therefore lag every fundamental figure by 90 days: the December quarter only becomes usable at the end of March. This is called **point-in-time** data — data as it was actually known at the time, rather than as it is known now.

Survivorship. If you build a list of companies from today’s index and then look at their history, you have accidentally selected for companies that survived. Every firm that went bankrupt is missing, so the historical returns look far too good. The fix is to build the universe from what existed at each date.

4 Making companies comparable

There is a practical problem with the panel. Book-to-market is a ratio around 0.3; market capitalization is a number in the billions; sales growth is a small percentage. A model has no way of knowing that a 0.1 change in one means something quite different from a 0.1 change in another. And the whole market drifts: valuations in 2021 were high across the board, so a book-to-market of 0.3 meant something different in 2021 than in 2009.

Both problems are solved by **standardizing within each month**.

A **z-score** rescales a number by asking “how many standard deviations from the average is it?”:

$$z = \frac{x - \text{mean}}{\text{standard deviation}}.$$

A z-score of 0 is exactly average, +2 is well above, −2 is well below. Because the units cancel, z-scores from completely different quantities are comparable.

We compute the mean and standard deviation *within each month, across companies* — a **cross-sectional** z-score. So the model never sees “book-to-market = 0.3”. It sees “this company is 0.8 standard deviations cheaper than its peers were that month”. The general level of the market cancels out automatically.

This is why the model is a *ranking* model almost by construction: after standardization, all it can see is each company’s position relative to its contemporaries.

Part II Training

5 What “training” actually means

Training (or *fitting*) a model means: show it many historical examples where both the features and the true answer are known, and let it adjust its internal settings until its predictions on those examples are as close as possible to the truth.

That is all it is. There is no understanding involved. The model is searching for a function — a rule that turns 66 standardized numbers into one score — that happened to work well on the past.

The kind of function we use is a **gradient-boosted decision tree ensemble**, which sounds forbidding and is genuinely simple once unpacked. Three ideas stack up.

A **decision tree** is a sequence of yes/no questions ending in a number. “Is book-to-market above 0.4? If yes, is return on equity above 0.1? If yes, predict +0.03.” Drawn out, the branching looks like a tree. An **ensemble** is many models whose answers are combined. One tree is crude; hundreds of trees, added together, can express a subtle rule.

Gradient boosting is a specific way of building the ensemble: rather than building all the trees independently, each new tree is trained to correct the errors the previous trees are still making. The first tree captures the strongest pattern, the second cleans up what the first got wrong, and so on. “Gradient” refers to the calculus used to work out which direction “wrong” points in.

We use an implementation called **LightGBM**, which is simply a fast, well-tested version of this idea. The research loop can also fit five other model types — ridge regression, elastic net, random forests and two dimension-reduction methods — for reasons we come to in Part IV.

A **parameter** is something the model learns from the data — where each tree puts its split points, what number each branch predicts.

A **hyperparameter** is something *you* choose before training starts: how many trees, how deep each one may be, how strongly to penalize complexity. The model cannot learn these; they are settings of the learning procedure itself. Choosing them well is most of what “tuning a model” means, and it is a central part of what the agents in Part IV argue about.

6 The thing that goes wrong: overfitting

Here is the central difficulty of the entire field.

A model with enough flexibility can fit *any* historical data perfectly — including the parts that were

pure luck. Stock returns are overwhelmingly noise; the predictable component is tiny. A sufficiently complicated model will happily memorize the noise, achieve a beautiful score on the data it was shown, and be worthless on anything new.

Overfitting is learning the accidents of your particular sample instead of the underlying pattern. The give-away is a large gap between performance on data the model was trained on and performance on data it has never seen.

Everything that follows in this Part is machinery for detecting overfitting honestly.

We measure this directly

Our reports carry a “train-minus-out-of-sample gap”. On a controlled test, a deliberately over-flexible model scored a perfect 1.000 on its training data and 0.133 on fresh data — a gap of 0.87, i.e. it had memorized. A properly regularized model scored 0.34 and 0.27: a gap of 0.08, i.e. it had learned something real. Note that the memorizing model looked *far better* on the training data. That is exactly why one number is never enough.

7 Splitting the data honestly

The defence against overfitting is to hold data back.

The panel is cut into three consecutive time periods:

- **Training set** — the model learns from this.
- **Validation set** — used to decide *when to stop* learning, and to compare candidate settings. The model does not learn from it directly, but choices are made using it, so it is partly used up.
- **Test set** — touched only at the very end, to get an honest score.

Because our data has a time dimension, the split is *chronological*: train on the earliest period, test on the latest. Splitting randomly would let the model train on 2023 and be tested on 2019, which is not a situation any real user is ever in.

Early stopping uses the validation set: keep adding trees, and each time, check whether the score on the validation data is still improving. When it stops improving, stop adding trees — further trees are fitting noise. This is the single most effective guard against overfitting in a boosted model.

The words **in-sample** and **out-of-sample** recur constantly. *In-sample* means measured on data the model was trained on — always flattering, never trustworthy. *Out-of-sample* means measured on data the model never saw. Only out-of-sample numbers are evidence.

8 The subtle leak: why a gap is needed

Now a problem specific to predicting the future, and one the pipeline originally got wrong.

Suppose the label is the return over the next *twelve* months. Take a training example from January 2023. Its label is the return from January 2023 to January 2024. To know that label at all, we must have data through January 2024.

So if the training set ends in January 2023 and the validation set begins in February 2023, the model has effectively been shown information running through January 2024 — which overlaps almost the entire validation period. The two sets are not independent. The model will look far better than it is.

An **embargo** (or **purge**) is a deliberate gap between the training period and the evaluation period, at least as long as the label horizon. With a twelve-month label, a model predicting June 2024 may only be trained on data whose labels were fully realized by June 2023.

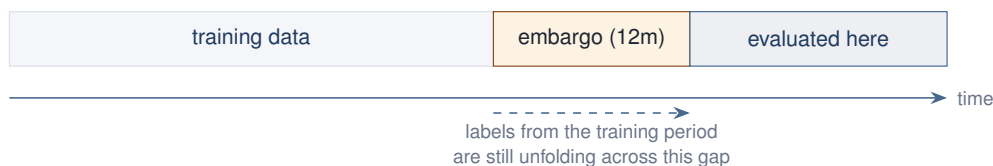


Figure 1. The embargo. Without the amber gap, the last training examples carry labels that reach into the evaluation window, and the two periods share outcomes.

This is not a small correction

Measured properly — with the embargo in place — the shipped twelve-month US model’s skill on the baseline settings is *negative*. Measured the old way, without a gap, it looked positive. Nothing about the model changed; only the honesty of the measurement did.

9 Walk-forward: the closest thing to a rehearsal

A single train/test split gives you one number from one arbitrary cut point. A better procedure imitates what actually happens in use.

In a **walk-forward** (or *expanding-window*) evaluation you repeat the following, marching through history: train on everything up to a cutoff, predict the next period, record the prediction, move the cutoff forward, retrain, and continue.

Every prediction is therefore made by a model that could genuinely have existed at that moment. Stitching them all together gives a track record rather than a single snapshot.

We retrain annually, which is both realistic — nobody refits a production model every month — and about twelve times cheaper than refitting monthly. Combined with the embargo, this is the evaluation every number in a validation report comes from.

Part III Judging a model

10 Skill

The **information coefficient (IC)** is the correlation, within a single month, between what the model predicted and what actually happened. One number per month.

The **rank IC** is the same thing computed on *ranks* rather than raw values (a Spearman rather than Pearson correlation): rank the predictions 1, 2, 3, ..., rank the realized returns 1, 2, 3, ..., and correlate those. It is our primary measure, for two reasons. It is unaffected by a handful of extreme returns, and it measures exactly what we care about — the ordering.

A rank IC of 1.0 would be a perfect ordering; 0 is no information; negative means the model is ordering backwards.

Calibrate your expectations

Stock returns are mostly noise, and the numbers here are much smaller than newcomers expect. A monthly-equity rank IC of about 0.03 is a *genuine, valuable* edge. 0.05 is strong. Anything above roughly 0.10 out-of-sample on real data is, in practice, almost always a bug or a leak rather than a discovery — which is why the reports place so much weight on whether the measurement was done honestly.

Consistency matters as much as size. A model with rank IC of 0.03 every single month is far more valuable than one averaging 0.03 by way of +0.4 and −0.34.

The **information ratio of the IC (ICIR)** divides the average IC by its standard deviation across months. It answers: relative to how much it bounces around, how big is the average?

One caution. The raw ratio is often quoted alongside an annualized figure, which is $\sqrt{12}$ times larger for monthly data — about $3.46\times$. Both appear in our reports, labelled, because comparing one against the other silently would flatter or penalize a model by a factor of three and a half.

R^2 **out-of-sample** asks what fraction of the variation in returns the model explained. In return prediction it is conventionally measured against a forecast of *zero* rather than against the historical average, because the historical average is itself an estimate and benchmarking against it flatters the model. Both versions are reported. Expect small numbers: an out-of-sample R^2 of 0.005 is respectable.

11 Saying how sure we are

A rank IC of 0.03 measured over 20 months and one measured over 200 months are not the same claim, but written down they look identical. So every headline figure carries an interval.

A **confidence interval** is a range that plausibly contains the true value. We compute ours by **bootstrapping**: take the list of monthly ICs, draw a new list of the same length by sampling from it at random with replacement, average it, and repeat a few hundred times. The spread of those averages shows how much the answer depends on which particular months you happened to observe.

An interval containing zero means “no result”

If the 95% interval runs from -0.01 to $+0.04$, then a true skill of zero is entirely consistent with what we observed. Our model validation agent treats this as disqualifying regardless of how good the central estimate looks, and blocks the model from being promoted. On the live twelve-month US model this is currently the binding constraint: the honest interval spans zero, so nothing is promoted.

12 Beyond skill: is it usable, and is it really alpha?

A statistically real signal can still be worthless. Three further checks.

Is the ordering real all the way through? Sort the companies into ten equal buckets by prediction — **deciles** — and look at the average realized return of each. If the model works, returns should rise steadily from bucket 1 to bucket 10. The **decile spread** is top bucket minus bottom bucket. But a large spread can come from a handful of extreme names while the middle is noise, so we also measure **monotonicity**: does the average return actually increase step by step? A large spread with poor monotonicity is a much weaker claim than it appears.

Can you trade it? **Turnover** is the fraction of the portfolio that has to be replaced each month. Every replacement costs money in commissions and in moving the price against yourself. A signal whose gross spread is 40 basis points a month against 80% turnover is, after costs, roughly nothing.

Is it alpha, or is it a disguise? This is the deepest of the three.

Decades of research have identified a handful of broad, well-known drivers of stock returns, called **factors**. The standard set — the **Fama–French** factors, after the economists who documented them — includes the overall market, company size (small firms behaving differently from large), value (cheap versus expensive), profitability, investment, and momentum (recent winners continuing to win).

A **beta** is the sensitivity of something to one of these factors. If your strategy has a large size beta, it goes up when small companies do well.

You can buy exposure to these factors extremely cheaply through index products. So if our model’s returns turn out to be nothing more than “it likes small cheap companies”, it is not worth running

— you could buy that exposure directly for a few basis points. We check by regressing the strategy’s returns on the six factors: the part explained by the factors is the disguise, and the leftover intercept is the real alpha. If that intercept is statistically indistinguishable from zero while the regression explains most of the variation, the honest conclusion is that there is no alpha here.

13 Breaking it on purpose

Everything so far assumes the data is correct. In reality our data comes from regulatory filings parsed by machine, and things go wrong: a filing arrives late, a figure is scaled wrongly, a company’s classification changes. A model that collapses when a small fraction of its inputs are damaged is dangerous, because bad-data days are exactly the days you most need it to behave.

So we damage the data deliberately and see what survives. The model is held fixed; only its inputs are degraded.

We simulate	Which stands for
Values go missing	A filing arrives late, or the parser fails on it.
Values are halved	A units or concept-mapping error — thousands read as millions.
Everything one month stale	The point-in-time alignment slips.
Newest month withheld	Reporting delay at the live edge.
Extreme returns injected	Corporate actions, or a thin stock blowing up.

For each, we measure how much skill was lost, and aggregate to a single rating on a three-point scale: **1 robust**, **2 moderate**, **3 fragile**. We take the *worst* case rather than the average, because someone choosing whether to trust a model needs the floor, not the typical outcome.

One complication. Suppose damage appears to hurt a lot — but the damage happened to land mostly on banks, and banks are simply harder to predict anyway. Then we have mistaken a property of the industry for a property of the perturbation. A variable that distorts a comparison this way is a **confounder**.

We correct for it by **stratifying**: measure the effect *separately within each industry* and then average those, so every industry counts equally and no industry’s share of the sample can drive the answer. The report also states how much of the raw effect turned out to be composition rather than damage.

What the rating catches that a skill score cannot

In a controlled test, an over-flexible model posted a skill score more than twice as high as a regularized one — and rated 3 (fragile) against 1 (robust). It had memorized, so small realistic defects destroyed it. Ranked on skill alone you would have chosen exactly the wrong model.

14 Where does it work?

An average over five thousand very different companies can hide a great deal, including a change of sign. So skill is broken out two ways, which answer different questions.

By industry. Companies are classified under **GICS** (the Global Industry Classification Standard, the standard scheme dividing listed companies into sectors and industry groups). This answers “*where* does this work?” in the terms a portfolio manager allocates by.

By factor exposure. Companies are sorted into fifths by their sensitivity to each Fama–French factor. This answers “*what kind of company* does this work on?” — and if the skill lives entirely in the smallest fifth, that is a size bet wearing a costume, whatever industries those companies happen to sit in.

A real example

On the live US twelve-month model, the baseline settings score -0.013 overall — slightly worse than useless. Broken out by industry, the same model scores $+0.040$ in Industrials. Both facts are true. Neither is visible in the headline, and the second one is where a researcher would start looking.

Buckets containing very few companies are computed but flagged, because a headline resting on eleven names is a curiosity, not a finding.

Part IV Agentic training

15 Why the old way was not enough

Until recently, training worked like this: run one fit, with one fixed set of choices, and whatever came out became the production model.

The choices were all reasonable. But nobody revisited them, nothing checked whether the resulting number was trustworthy, and there was no record of *why* the model looked the way it did. When the desk’s screen reported skill of 0.001 — its own label for which read “negligible” — nothing in the system reacted.

Worse, some of the fixed choices were quietly harmful, and two in particular:

- Missing values were filled with the month’s *average*, and there was no limit on how sparse a feature could be before it was used anyway.
- Extreme returns were never trimmed during training, although the evaluation code trimmed them.

Changing just those two — filling with the *median* instead, and trimming the most extreme returns — moved the twelve-month US model from -0.013 to $+0.017$. Neither change is clever. Neither would ever have been tried by a procedure with no step in which somebody asks “is this actually working?”.

That step is what we added.

16 What an “agent” is here

A **large language model (LLM)** is a system trained to continue text — the technology behind ChatGPT and its relatives. An **agent**, in this codebase, is an LLM given three things: a *role* (who it is and what it cares about), a *packet of information* (this round’s results), and a *required output format* it must answer in.

That last constraint matters. The agents do not write prose that a human then interprets. Each returns a filled-in form with named fields — a proposed change, a verdict, a list of findings — which the surrounding program reads directly. This is called **structured output**.

Four agents, deliberately modelled on how a real research desk argues:

Role	Asks	Can
Quantitative Researcher	What should we change next, and why?	Propose the next settings
Model Validation	Is this result even believable?	<i>Veto</i> a model
Portfolio Manager	Would I put money behind this?	End the search; approve promotion
External Advisor	What is everyone here failing to see?	Advise only

The External Advisor is deliberately run on a *different* AI provider from the other three whenever a second key is available. The point is not redundancy but independence: a different model brings different habits, and the role exists to say the unwelcome thing.

17 LangGraph: wiring the agents together

LangGraph is a library for building workflows as a **graph**: a set of **nodes** (steps) joined by **edges** (what runs next). All the steps read from and write to one shared dictionary called the **state**.

Three features earn its place here:

- **Branching.** An edge can be conditional — a function looks at the state and decides where to go next. This is how the loop decides to run another round or stop.
- **Parallelism.** Several nodes can run in the same step. Our three critics do, since none of them depends on the others.
- **Reducers.** If two nodes running at the same time both write to the same field, something must decide how to combine them. A **reducer** is that rule. Ours simply appends to a list, so all three critics' opinions are kept rather than the last writer overwriting the rest.

The same library already runs the investment committee elsewhere in this application, so the pattern is familiar to the codebase.

18 One round, step by step

1. **Load the data.** Done once and reused — it is the slow part.
2. **Train.** Apply this round's settings and fit a model.
3. **Score it.** Purged walk-forward, then the full battery from Part III.
4. **Try to break it.** The perturbations, and the robustness rating.
5. **Write the report.** Everything above, assembled into one document.
6. **The critics read it, simultaneously.** Validation looks for reasons to disbelieve it. The Portfolio Manager asks whether it is worth trading. The Advisor argues the other side.
7. **The researcher responds** — reading all three critiques — and proposes one coherent change, with a prediction of what it should do.
8. **Continue or stop.**

The loop stops for any of five reasons: the round budget is used up; the Portfolio Manager accepts or rejects; the researcher declares the search finished; the researcher proposes nothing valid; or the score has not improved for two consecutive rounds. The reason is recorded, so the report says why it ended rather than leaving you to guess.

19 Two things the AI is not allowed to do

An LLM steering a training pipeline is a genuine risk, and two specific dangers are closed off structurally rather than by asking nicely.

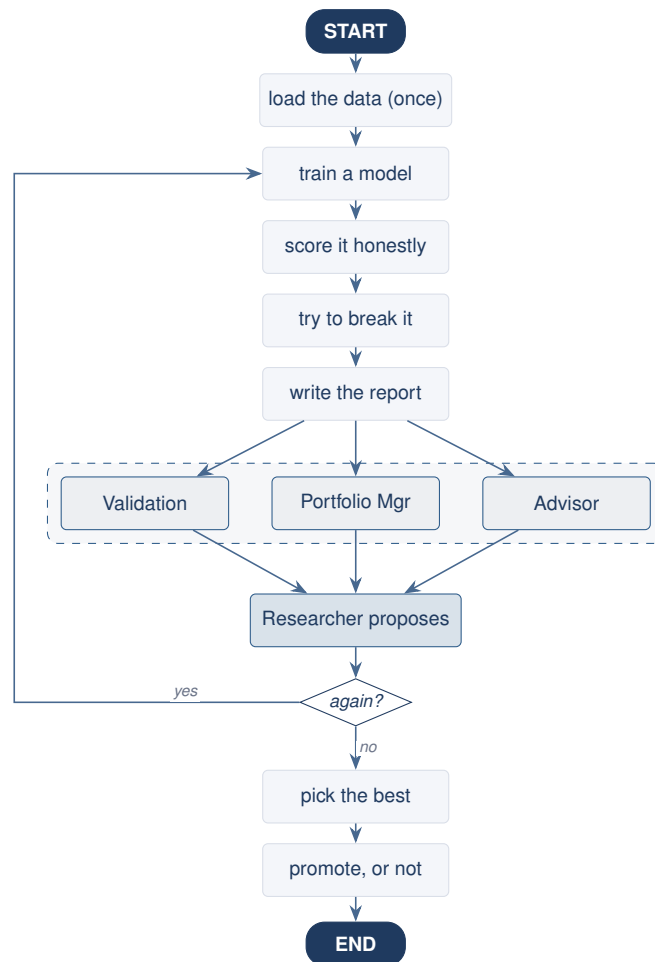


Figure 2. One round of the loop. The three critics run at the same time; the researcher waits for all of them before proposing the next change.

It cannot run code

The researcher does not write code and never touches the data. It fills in a form listing which settings to change. A separate, ordinary piece of program logic then checks that form: every field name must be one of the known settings, every value must lie in an allowed range, and hyperparameters belonging to a different model type are discarded. Anything rejected is reported back to the agent in the next round.

So a proposal of “set the learning rate to 500” becomes “learning rate clamped to the maximum”, and a proposal naming a setting that does not exist is simply dropped. The set of things the agents can actually cause is exactly the set we specified.

It cannot see which companies are which

This one is subtler and matters more.

Why the agents are never shown company names

The AI models we use were trained on text from the internet, which includes financial news up to some cutoff date. That cutoff is *after* the period we evaluate on. So an agent shown the names of the best-performing stocks in our test window may “recognize” them — not from the diagnostics, but from memory.

It could then steer the settings toward those companies. The resulting skill score would be real, reproducible, and completely worthless, because the model would be tuned to a future the agent already knew.

The packet handed to an agent therefore contains aggregate statistics only: skill numbers, industry-level summaries, counts. No company names, no per-company returns, no ranked lists. A guard checks every packet against the actual list of companies in the run before it is sent.

A detail that mattered in practice

The first version of that guard looked for anything *shaped* like a stock symbol. It failed immediately on real data, because many genuine US tickers are ordinary English words — ALL, ON, MAX, GAP, REAL, CARE. Almost every sentence an agent wrote tripped it. The working version checks against the run's real list of companies, and only in places where an identifier could actually sit — not inside prose.

20 Choosing a winner, and when not to

At the end, the surviving candidates compete — along with a **combination** of them.

An **ensemble** here means averaging several models' rankings. It often beats every individual model, because each one's errors are partly independent, so averaging cancels some of the noise while keeping the shared signal. The loop produces its candidates anyway, so testing the combination costs nothing extra.

The winner is promoted to production *only* if all of the following hold:

- it beats the model currently in production, by a margin, measured out-of-sample;
- the Model Validation unit did not veto it;
- the Portfolio Manager approved;
- it is not rated fragile.

Otherwise the live model is left alone, the candidate is saved separately, and the report states which condition failed. When every round is vetoed, the run still names its best attempt and marks it unpromotable — more useful than reporting nothing.

An honest caveat we print on every run

The model currently in production had its score measured the old way — one split, no embargo. The challenger's score is measured the new, stricter way. The comparison therefore favours the incumbent, and the reports say so, because quietly comparing a lenient number against a strict one would be the easiest way to reach a wrong conclusion.

21 What you get out

Every round produces a validation report, and every run produces a downloadable dossier: the skill numbers with their intervals, the robustness rating and what broke it, the industry and factor-exposure breakdowns, which features mattered and whether that ranking is stable, and all four agents' reasoning in their own words.

The point is not that the machine found the answer. It is that the reasoning is written down — what was tried, what it scored, who objected, and why the thing that shipped (or did not ship) was chosen.

Glossary

Term	Meaning
Agent	An LLM given a role, a packet of data, and a required answer format.
Alpha	Return not explained by exposure to known market factors.
Beta	Sensitivity of a return stream to one such factor.
Bootstrap	Estimating uncertainty by resampling the data you already have.
Confounder	A third variable that distorts an apparent relationship.
Cross-sectional	Comparing many companies at one moment (vs. one company over time).
Decile	One tenth of the sorted list.
Early stopping	Halting training when validation performance stops improving.
Embargo (purge)	A deliberate gap between training and evaluation periods.
Ensemble	Several models combined into one prediction.
Factor	(i) an input feature; (ii) a known broad driver of returns, e.g. size, value.
Feature	One measured company property used as model input.
Fama–French factors	The standard set: market, size, value, profitability, investment, momentum.
GICS	The standard scheme classifying companies into sectors and industries.
Gradient boosting	Building an ensemble where each model corrects its predecessors' errors.
Hyperparameter	A setting chosen before training, not learned from data.
IC	Information coefficient: correlation between prediction and outcome.
ICIR	Average IC divided by its standard deviation — consistency of skill.
In-sample	Measured on data the model trained on. Never trustworthy alone.
Label	The quantity being predicted; here, forward return.
LangGraph	A library for building workflows as graphs of steps over shared state.
Lookahead bias	Accidentally using information not available at the time.
Monotonicity	Whether returns rise steadily across prediction deciles.
Out-of-sample	Measured on data the model never saw. The only real evidence.
Overfitting	Learning the noise in your sample rather than the pattern.
Panel	A table indexed by both entity and time.
Point-in-time	Data as it was actually known at the historical date.
Rank IC	IC computed on ranks; robust, and matches what we care about.
Reducer	The rule for merging concurrent writes to a shared field.
R^2 out-of-sample	Fraction of variation explained, on unseen data.
State	The shared dictionary all graph steps read and write.
Structured output	Requiring the LLM to answer in a fixed schema.
Supervised learning	Learning from examples where the answer is known.
Turnover	The share of the portfolio replaced each period.
Winsorizing	Replacing extreme values with less extreme ones instead of deleting them.
z-score	$(x - \text{mean})/\text{standard deviation}$; puts different quantities on one scale.

The formal statements live in the *Quantitative & ML Models* volume; the implementation in the *glib Integration* volume. If a term here still feels slippery, that is worth saying — the point of this volume is that none of it should.